Informe de calidad del Sprint 2

App Bicis · Grupo 3 · Proyecto Integrado 2026

Metadato	Valor
Grupo / Sprint	Grupo 3 / Sprint 2
Fecha de realización	05/11/2026
Análisis de SonarCloud	Rama **develop**, 05/11/2026 10:52 (organización isuc, proyecto App-Bicis-Grupo3, Quality Gate "ISUC")
Commit analizado	**a0f40a1d7822713f5f9874bf977e10d996248b9c**
Contenido del commit	Merge de la PR #36 (actualización del manual de usuario) en develop. Incluye las tres historias del Sprint 2: 532335 Filtrar por huecos, 532304 Filtrar por distancia y 532314 Buscar estación por nombre

1. Estado del producto

1.1 Resumen del análisis

El Quality Gate **se supera**. Security, Reliability y Maintainability tienen nota **A** (Figura 1). La Tabla 1 recoge cada condición del Quality Gate ISUC con su valor actual.

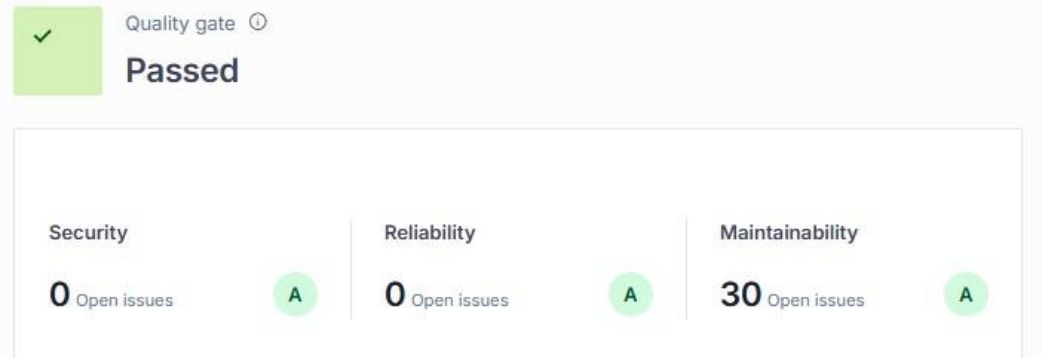


Figura 1. Resumen del análisis de develop (commit a0f40a1): Quality Gate superado.

Condición (Quality Gate ISUC)	Umbral	Valor	Estado
Reliability rating	A	A (0 bugs)	Cumple
Security rating	A	A (0 vulnerabilidades)	Cumple
Maintainability rating	A	A (30 issues)	Cumple
Líneas duplicadas	≤ 60 %	0,0 %	Cumple
Ratio de deuda técnica	≤ 10 %	0,2 %	Cumple
Deuda técnica	≤ 250 min	**237 min**	Cumple, 13 min de margen

Tabla 1. Condiciones del Quality Gate ISUC en develop (SonarCloud, Summary y API del Quality Gate).

1.2 Severidad de los issues de cada elemento

Security y Reliability no tienen ningún issue (Figura 2). Los 30 issues abiertos son de Maintainability: **1 High, 18 Medium, 8 Low y 3 Info**, ninguno Blocker (Figura 3).

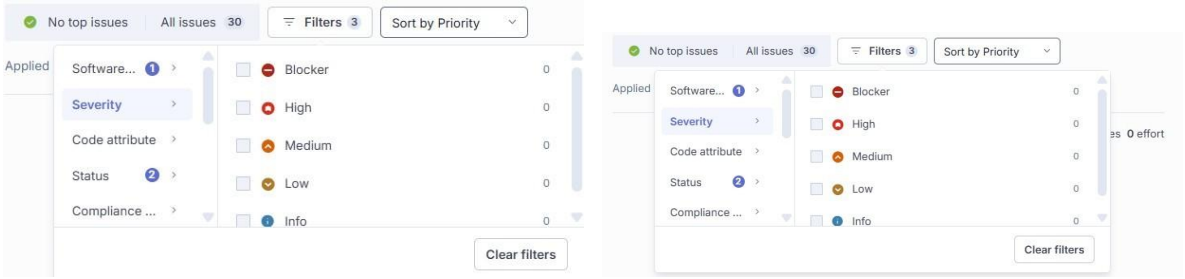


Figura 2. Severidad de los issues de Security (izquierda) y Reliability (derecha): 0 en todas las severidades.

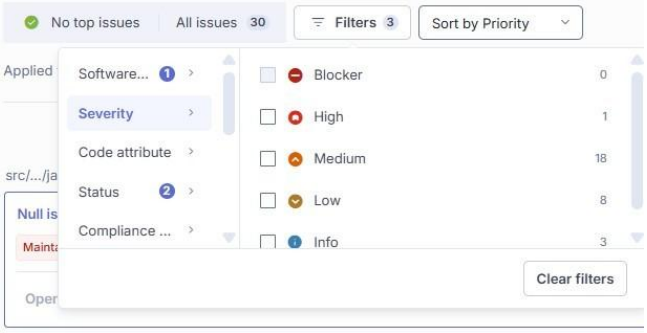


Figura 3. Severidad de los issues de Maintainability: Blocker 0, High 1, Medium 18, Low 8, Info 3.

1.3 Evolución

La Figura 4 muestra el número de issues por elemento. Security y Reliability se mantienen en 0 en todo el periodo; Maintainability sube hasta 34 al final del Sprint 1 y baja a 30 en el Sprint 2. La Figura 5 muestra que no hay código duplicado (0,0 %) y que la complejidad ciclomática crece de forma continua. La Tabla 2 resume los valores de cada análisis de develop.





Figura 4 (izquierda). Evolución del número de issues de Security, Reliability y Maintainability. Figura 5 (derecha). Evolución del código duplicado (%) y de la complejidad ciclomática.

Análisis develop	Líneas	Issues	Deuda (min)	Complejidad	Duplic.	Cobertura
08/10	912	12	54	56	0 %	1,4 %
22/10	1.770	23	111	197	0 %	44,5 %
26/10 (fin Sprint 1)	2.265	34	219	280	0 %	49,1 %
03/11	2.426	30	224	280	0 %	48,6 %
04/11	2.810	32	224	346	0 %	47,7 %

05/11 (actual)	**3.405**	**30**	**237**	**477**	**0 %**	**47,3 %**

Tabla 2. Evolución de las métricas de develop (SonarCloud, Activity).

1.4 Burndown Chart

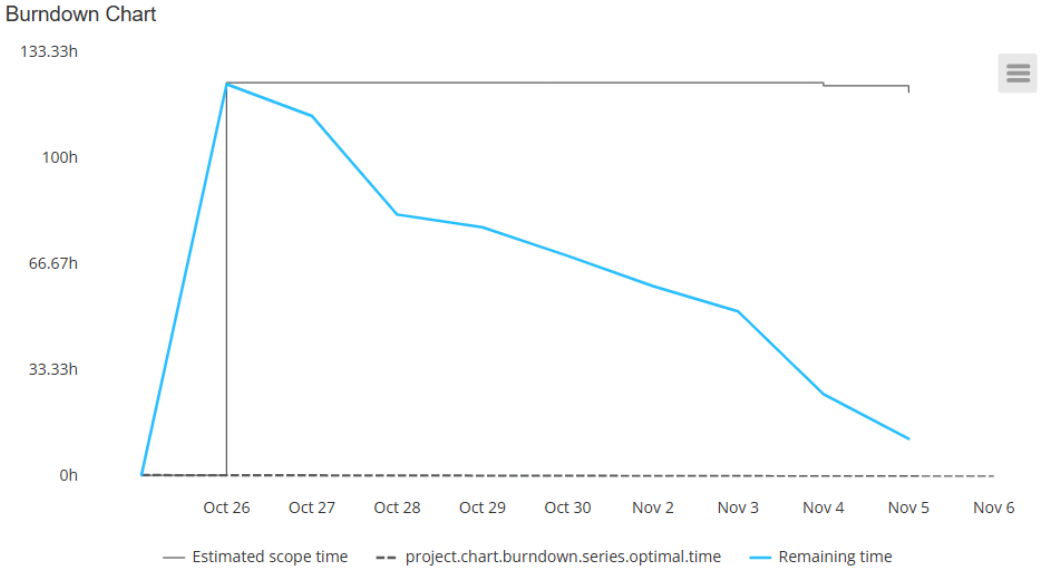


Figura 6. Burndown Chart del grupo en ScrumDesk (Reports, vista Time).

2. Análisis

El estado del producto es bueno. El Quality Gate se supera, los tres elementos tienen nota A, no hay bugs, vulnerabilidades ni código duplicado, y todos los issues abiertos son de mantenibilidad (Tabla 1, Figuras 1 a 3). Respecto al Sprint 1 se ha corregido el bug de fiabilidad que hacía fallar el Quality Gate (java:S2184 en `DeviceLocationProvider`) y los tres issues High de `StationsView` (java:S3252) (Tabla 4).

El problema principal es que la deuda técnica está casi en el límite. El Quality Gate ISUC exige como mucho 250 minutos de deuda y develop tiene **237 minutos**: quedan solo 13 minutos de margen (Tabla 1). Unos pocos issues nuevos de 5 o 10 minutos en el Sprint 3 bastarían para que el Quality Gate falle y develop quede bloqueado.

Causas. En el Sprint 2 el código ha crecido un 50 % (de 2.265 a 3.405 líneas) con las tres historias nuevas. El número de issues ha bajado de 34 a 30, pero la deuda ha subido de 219 a 237 minutos porque los issues que quedan son más caros de corregir (Tabla 2, Figura 4). La deuda está muy concentrada: cinco grupos de issues suman 180 de los 237 minutos, el 76 % (Tabla 3). El mayor bloque son los 4 usos de `Thread.sleep()` en los tests de interfaz (80 minutos), que además hacen los tests lentos e inestables en el emulador. Los tres issues java:S2589 de `StationsView` son nuevos de este sprint: aparecen en el botón Aplicar del diálogo de filtros que se amplió para los filtros por huecos y por distancia.

Issue	Regla	Dónde	Deuda
Uso de *Thread.sleep()* en tests	java:S2925	3 tests de Detectar ciudad y *MockLocationManager* (×4)	80 min
Devuelve null en vez de un array vacío	java:S1168	*StringArrayAdapter*	30 min
Expresión que siempre vale true	java:S2589	*StationsView* (×3, líneas 505, 508 y 511)	30 min

Devuelve null donde se espera un Boolean (único High)	java:S2447	*BooleanAdapter*	20 min

Issue	Regla	Dónde	Deuda
Falta el constructor privado	java:S1118	*Utils*, *CityBikesService*, *LocationModule*, *RepositoriesModule* (×4)	20 min

Tabla 3. Issues con más deuda técnica en develop (SonarCloud, Issues, ordenados por prioridad).

Complejidad y cobertura. La complejidad ciclomática ha subido un 70 % en el sprint (de 280 a 477, Figura 5), algo esperable porque se han añadido filtros combinados, búsqueda con sugerencias y diálogos, pero hay que vigilarla. La cobertura baja ligeramente, del 49,1 % al 47,3 %; no forma parte del Quality Gate.

Causa de proceso. Como en el Sprint 1, las historias se integraron en develop en la segunda semana (la primera historia del Sprint 2 entró en develop el 03/11), así que los issues nuevos se han detectado con poco margen para corregirlos. Además, la deuda arrastrada del Sprint 1 no se ha reducido porque las acciones 3 a 7 de su plan siguen pendientes (Tabla 4).

2.1 Seguimiento del plan del Sprint 1

#	Acción del plan del Sprint 1	Regla	Estado (05/11)
1	Multiplicación en long en *DeviceLocationProvider*	java:S2184	Hecho
2	Constantes desde la clase que las declara en *StationsView*	java:S3252 ×3	Hecho
3	Boolean no nulo en *BooleanAdapter*	java:S2447	Pendiente
4	Constructores privados en utilidades y módulos	java:S1118 ×4	Pendiente
5	Sustituir *Thread.sleep()* en los tests	java:S2925 ×4	Pendiente
6	Array vacío en *StringArrayAdapter*	java:S1168	Pendiente
7	Limpieza rápida (imports, código comentado, variable sin usar, modificadores)	varias	Pendiente

Tabla 4. Estado de las acciones del plan de calidad del Sprint 1 en el análisis actual.

3. Plan de acción

Criterio de prioridad: primero el único issue High, después los issues nuevos de este sprint y, por último, las acciones que más deuda quitan por minuto invertido, para recuperar margen frente al umbral de 250 minutos. Las acciones 1 a 4 quitan **150 minutos** de deuda (de 237 a unos 87) con unas 2,5 horas de esfuerzo según SonarCloud, que en ScrumDesk hemos estimado en 3,5 h (Tabla 6), así que caben en lo que queda de sprint, antes de preparar la release. Lo hemos mirado con el Burndown Chart (Figura 6): el 05/11 quedaban unas 12 horas pendientes y el sprint acaba el 06/11, pero varios miembros ya habían terminado sus tareas de las historias, así que esas 3,5 h se podían asumir sin retrasar nada. Las acciones 5 y 6 se planifican al inicio del Sprint 3.

#	Acción	Regla y fichero	Esfuerzo	Cuándo
1	Devolver un Boolean no nulo o cambiar el contrato del adaptador	java:S2447 · *BooleanAdapter*	20 min	Sprint 2

2	Sacar la validación de los valores del diálogo a un método que devuelva si son válidos, en lugar de comprobar un flag justo después de ponerlo a false	java:S2589 ×3 · *StationsView*	30 min	Sprint 2
3	Sustituir *Thread.sleep()* por esperas con condición (comprobar el foco o el texto esperado cada 100 ms con tiempo máximo), como ya se hace en *StationsUITestUtils*	java:S2925 ×4 · tests de Detectar ciudad y *MockLocationManager*	80 min	Sprint 2
#	**Acción**	**Regla y fichero**	**Esfuerzo**	**Cuándo**
4	Añadir constructores privados a las clases de utilidades y módulos	java:S1118 ×4 · *Utils*, *CityBikesService*, *LocationModule*, *RepositoriesModule*	20 min	Sprint 2
5	Devolver un array vacío en lugar de null	java:S1168 · *StringArrayAdapter*	30 min	Inicio Sprint 3
6	Limpieza rápida: imports duplicados o sin usar, código comentado en XML, variable sin usar en *ExampleTest*, orden de modificadores y argumentos de *assertEquals*	java:S1128 ×3, xml:S125 ×2, java:S1854, java:S1481, java:S1124, java:S3415 ×2	≈ 25 min	Inicio Sprint 3

Tabla 5. Plan de acción priorizado (esfuerzo estimado por SonarCloud; total ≈ 3 h 25 min).

Se dejan fuera los comentarios TODO (java:S1135, severidad Info y 0 minutos), los bloques anidados del adaptador de redes de la plantilla inicial (java:S1199) y la parametrización de `IntegerAdapterTest` (java:S5976): no ponen en riesgo el Quality Gate y su beneficio es menor que el de las acciones anteriores.

Acciones preventivas:

- Usar SonarQube for IDE (SonarLint) en Android Studio conectado a la organización isuc y no abrir una PR a develop con issues nuevos de severidad High o Medium.
- Al revisar cada PR, comprobar en SonarCloud que la deuda técnica de develop sigue por debajo de 200 minutos; si la supera, la PR incluye la corrección.
- No usar `Thread.sleep()` en los tests nuevos; usar las esperas con condición de `StationsUITestUtils`.
- Integrar en develop a mitad de la primera semana del Sprint 3 para tener un análisis temprano y hacer el informe de calidad al final de esa semana, como pide la guía.

4. Tareas añadidas al Sprint 2

ID ScrumDesk	Tarea	Estimación	Responsable
978448	Calidad SP2: corregir java:S2447 en BooleanAdapter (acción 1)	0,5 h	Diego
978449	Calidad SP2: corregir java:S2589 en StationsView (acción 2)	0,5 h	Diego

978451	Calidad SP2: sustituir Thread.sleep() en los tests de interfaz (acción 3)	2 h	Diego
978450	Calidad SP2: constructores privados java:S1118 (acción 4)	0,5 h	Diego

Tabla 6. Tareas del plan de acción añadidas al Sprint 2 en ScrumDesk (total 3,5 h).

Ejecución del plan. Las acciones 1 a 4 se hicieron el 05/11/2026 y se integraron en develop (commit c8254d4). Los Thread.sleep() se han cambiado por SystemClock.sleep() dentro de las esperas con condición, igual que en StationsUITestUtils. En BooleanAdapter hemos dejado que devuelva null, porque en la HU de información de la red el null significa "dato no disponible" y si devolviera false la app diría que la red no tiene bicis eléctricas sin saberlo; lo hemos dejado documentado en el código. En el nuevo análisis de SonarCloud el Quality Gate se sigue pasando, los issues de Maintainability bajan de 30 a 18 y la deuda técnica baja de 237 a 87 minutos, así que volvemos a tener margen frente al límite de 250 (Figura 7).

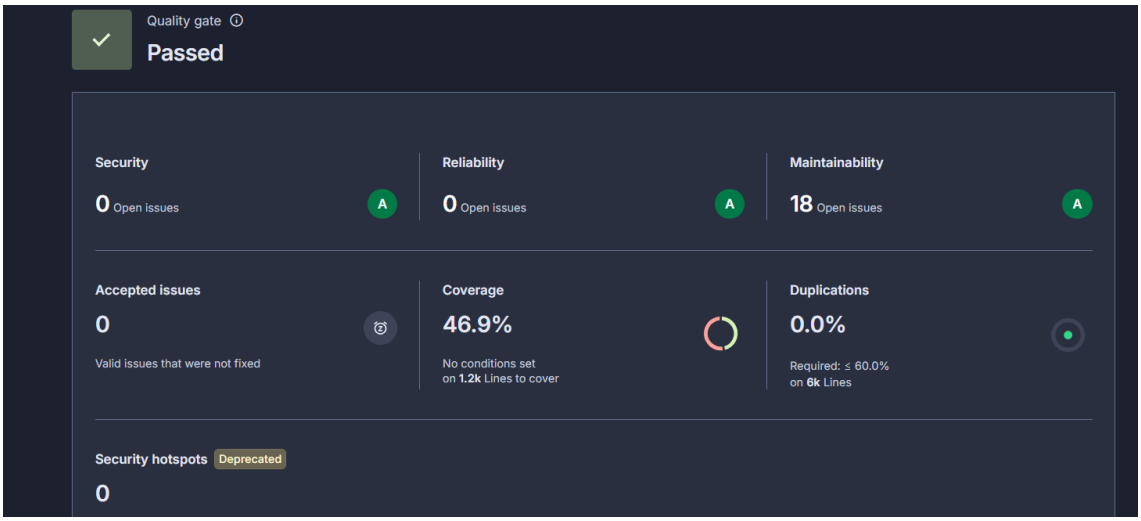


Figura 7. Resumen del análisis de develop después de aplicar el plan (commit c8254d4): 18 issues de Maintainability y 1 h 27 min de deuda técnica.